

Learn ESP32 for Home Automation

Lesson 4: Connecting ESP32 to Wi Fi

Connecting ESP32 to Wi Fi – Study Notes

Key Concepts

1. **Wi Fi library initialization** – Using `WiFi.h` (or `ESP8266WiFi.h` for ESP8266) and calling `WiFi.begin(ssid, password)`.
2. **Connection workflow** – Blocking vs. non blocking loops, timeout handling, and checking `WiFi.status()`.
3. **Network information retrieval** – Obtaining the device's IP address, subnet mask, gateway, and RSSI after a successful connection.
4. **Security considerations** – Storing credentials safely (e.g., `Preferences`, `ArduinoJson` encrypted storage, or compile time constants).
5. **Debugging tools** – Serial monitor messages, Wi Fi diagnostics (`WiFi.RSSI()`, `WiFi.scanNetworks()`) and LED indicators.

Summary

Connecting an ESP32 to a Wi Fi network is one of the first steps in most IoT projects. The process begins with including the `WiFi` library and supplying the network's SSID and password to `WiFi.begin()`. The board then attempts to associate with the access point, and the programmer typically monitors `WiFi.status()` until it reaches `WL_CONNECTED` or a timeout occurs. Once connected, the ESP32 receives a DHCP assigned IP address, which can be queried with `WiFi.localIP()`.

Beyond the basic connection, it's good practice to handle errors (e.g., wrong password, AP not found) and to keep credentials out of plain source code. Simple debugging can be done via the Serial console, while visual feedback (LED blink patterns) helps when a monitor isn't attached. After the connection is stable, the IP address and other network parameters become available for use in HTTP requests, MQTT, or other network services.

Important Points

- **Include the library:** `#include <WiFi.h>` (ESP IDF: `esp_wifi.h` + `esp_event.h`).
- **Start the connection:** `WiFi.begin(ssid, password);` – can also specify a static IP before `begin()` if DHCP isn't desired.

- **Check status:**

```
```cpp
while (WiFi.status() != WL_CONNECTED) {
 delay(500);
 Serial.print(".");
}
```

---

- **Timeout handling:** Use `millis()` to break out of the loop after a reasonable period (e.g., 20 /s).
- **Retrieve IP:** `IPAddress ip = WiFi.localIP();` `Serial.println(ip);``
- **Other info:**
  - `WiFi.subnetMask()`
  - `WiFi.gatewayIP()`
  - `WiFi.RSSI()` (signal strength)
    - **Secure storage:**
  - Hard code only for quick demos.
  - For production, store in **NVS** (`Preferences` library) or load from encrypted file/OTA config.
    - **Debug output:** Keep Serial at 115200 baud; print status, IP, and error codes.
    - **Power considerations:** After connection, you can call `WiFi.setSleep(true)` to reduce current draw.

---

## Examples

### Example 1 – Basic Wi Fi connection with serial feedback

```
```cpp
#include <WiFi.h>
const char* ssid    = "YourNetworkSSID";
const char* password = "YourNetworkPass";
void setup() {
  Serial.begin(115200);
  delay(1000);
  Serial.println("\nConnecting to Wi Fi...");
  WiFi.begin(ssid, password);
  unsigned long startAttemptTime = millis();
  const unsigned long timeout = 20000; // 20 /s
  while (WiFi.status() != WL_CONNECTED && millis() - startAttemptTime < timeout) {
    Serial.print(".");
    delay(500);
  }
  if (WiFi.status() == WL_CONNECTED) {
    Serial.println("\n Connected!");
    Serial.print("IP address: ");
    Serial.println(WiFi.localIP());
  } else {
    Serial.println("\nL Connection failed.");
  }
}
void loop() {
  // put your main code here, to run repeatedly
}
```

```
}  
...
```

Explanation: The sketch attempts to join the AP, prints dots while waiting, and either shows the assigned IP or reports failure after 20 /seconds.

Example 2 – Non blocking connection with LED indicator

```
```cpp  
#include <WiFi.h>
const char* ssid = "MySSID";
const char* password = "MyPass";
const int LED_PIN = 2; // Built in LED on many ESP32 dev boards
bool ledState = false;
unsigned long previousMillis = 0;
const long interval = 500; // blink every 500 /ms while connecting
void setup() {
 pinMode(LED_PIN, OUTPUT);
 Serial.begin(115200);
 WiFi.begin(ssid, password);
 Serial.println("Connecting...");
}
void loop() {
 // Blink while not connected
 if (WiFi.status() != WL_CONNECTED) {
 unsigned long current = millis();
 if (current - previousMillis >= interval) {
 previousMillis = current;
 ledState = !ledState;
 digitalWrite(LED_PIN, ledState);
 }
 } else {
 // Connected – turn LED solid ON and print IP once
 digitalWrite(LED_PIN, HIGH);
 static bool printed = false;
 if (!printed) {
 Serial.print("Connected! IP: ");
 Serial.println(WiFi.localIP());
 printed = true;
 }
 }
}
}```
```

\*Explanation:\* This sketch avoids a blocking `while` loop, allowing other tasks (e.g., sensor reads) to run while the ESP32 is still trying to join the network. The LED blinks until the connection succeeds, then stays on.

---

## Quick Review

1. **What function starts the Wi-Fi connection on an ESP32?**
2. **Which `WiFi` status value indicates a successful connection?**
3. **How can you retrieve the ESP32's assigned IP address?**
4. **Name two ways to protect Wi-Fi credentials in a production sketch.**
5. **Why might you prefer a non-blocking connection approach in a real-world project?**

---

## Further Reading

- **ESP-IDF Wi-Fi API** – deeper dive into event handling, power save modes, and custom Wi-Fi drivers.
- **Static IP configuration** – `WiFi.config()` usage.
- **Secure credential storage** – `Preferences` (NVS), `ArduinoJson` encrypted files, or OTA configuration servers.
- **Advanced networking** – MQTT, HTTP/HTTPS client libraries, and WebSocket integration.
- **Wi-Fi scanning & diagnostics** – `WiFi.scanNetworks()`, `WiFi.RSSI()`, and troubleshooting weak signals.

